Atividade: Use expressões regulares para encontrar padrões

Introdução

Analistas de segurança frequentemente analisam arquivos de log, incluindo aqueles que contêm informações sobre tentativas de login. Por exemplo, como analista, você pode sinalizar endereços IP relacionados a tentativas incomuns de acesso ao sistema.

Outra área de foco em cibersegurança é a detecção de dispositivos que necessitam de atualizações. As atualizações de software ajudam a prevenir problemas de segurança decorrentes de vulnerabilidades.

O uso de expressões regulares em Python pode ajudar a automatizar os processos envolvidos nessas duas áreas da cibersegurança. Padrões e funções de expressões regulares podem ser usados para extrair informações importantes de strings e arquivos de forma eficiente.

Neste laboratório, você escreverá expressões regulares para extrair informações como IDs de dispositivos ou endereços IP.

Dicas para concluir este laboratório

Cenário

Neste laboratório, você trabalhará como analista de segurança e suas principais tarefas serão as seguintes:

- Extração de IDs de dispositivos contendo determinados caracteres de um registro; esses caracteres correspondem a um determinado sistema operacional que requer atualização.
- Extrair todos os endereços IP de um registro e compará-los com aqueles que estão sinalizados em uma lista.

Tarefa 1

Para trabalhar com expressões regulares em Python, comece importando o `re` módulo. Este módulo contém diversas funções que o ajudarão a trabalhar com expressões regulares. Ao executar o seguinte código, o módulo estará disponível em todo o notebook.

Em [1]:
```
# Importe o módulo `re` em Python

importar  re
```

Tarefa 2

Atualmente, você está procurando por IDs de dispositivo que começam com `"r15"` . Esses caracteres indicam que o dispositivo está executando um sistema operacional que precisa ser atualizado.

Você recebeu um registro de IDs de dispositivos, armazenado em uma variável chamada `devices` . Seu objetivo final é extrair os IDs de dispositivos que começam com os caracteres `"r15"` . Por enquanto, exiba o conteúdo da string completa para examinar o que ela contém. Certifique-se de substituir o código `### YOUR CODE HERE ###` pelo seu próprio código antes de executar a próxima célula.

Em [3]:
```
# Assign `devices` to a string containing device IDs, each device ID repres
ented by alphanumeric characters

devices = "r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk
253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x482kt 6oa6m6u x3463ac i4l56nq g07
h55q 081qc9t r159r1u"

# Display the contents of `devices`

print(devices)
```

```
r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk 253be78 ac
742a1 r15u9q5 zh86b2l ii286fq 9x482kt 6oa6m6u x3463ac i4l56nq g07h55q 081q
c9t r159r1u
```

Hint 1

Task 3

In this task, you'll write a pattern to find devices that start with the character combination of `"r15"` .

Use the regular expression symbols `\w` and `+` to create the pattern, and store it as a string in a variable named `target_pattern` .

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell. Note that the code cell will contain only variable assignments, so running it will not produce an output.

In [5]:
```
# Assign `devices` to a string containing device IDs, each device ID repres
ented by alphanumeric characters

devices = "r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk
253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x482kt 6oa6m6u x3463ac i4l56nq g07
h55q 081qc9t r159r1u"

# Assign `target_pattern` to a regular expression pattern for finding devic
e IDs that start with "r15"

target_pattern = "r15\w+"
```

Hint 1

Hint 2

Hint 3

Question 1

What regular expression pattern did you use? For each component of the pattern, what would happen if it were missing?

O padrão r15\w+ usa r15 para filtrar o prefixo, \w para identificar caracteres alfanuméricos e + para capturar o ID completo; sem esses itens, o filtro falharia, não reconheceria os caracteres ou cortaria o código pela metade.

Task 4

Use the `findall()` function from the `re` module to find the device IDs that the `target_pattern` matches with. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

Note: In order to use `re.findall()` in Tasks 4, 7, 8, 9 and 11, you must have previously run the code `import re` in Task 1.

In [6]:
```
# Assign `devices` to a string containing device IDs, each device ID repres
ented by alphanumeric characters

devices = "r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk
253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x482kt 6oa6m6u x3463ac i4l56nq g07
h55q 081qc9t r159r1u"

# Assign `target_pattern` to a regular expression pattern for finding devic
e IDs that start with "r15"

target_pattern = "r15\w+"

# Use `re.findall()` to find the device IDs that start with "r15" and displ
ay the results

print(re.findall("r15\w+", devices))
```

```
['r151dm4', 'r15xk9h', 'r15u9q5', 'r159r1u']
```

Hint 1

Hint 2

Task 5

Now, the next task you're responsible for is analyzing a network security log file and determining which IP addresses have been flagged for unusual activity.

You're given the log file as a string stored in a variable named `log_file`. There are some invalid IP addresses in the log file due to issues in data collection. Your eventual goal is to use regular expressions to extract the valid IP addresses from the string.

Start by displaying the contents of the `log_file` to examine the details inside. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [7]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Display contents of `log_file`

print(log_file)
```

```
eraab 2022-05-10 6:03:41 192.168.152.148
iuduike 2022-05-09 6:46:40 192.168.22.115
smartell 2022-05-09 19:30:32 192.168.190.178
arutley 2022-05-12 17:00:59 1923.1689.3.24
rjensen 2022-05-11 0:59:26 192.168.213.128
aestrada 2022-05-09 19:28:12 1924.1680.27.57
asundara 2022-05-11 18:38:07 192.168.96.200
dkot 2022-05-12 10:52:00 1921.168.1283.75
abernard 2022-05-12 23:38:46 19245.168.2345.49
cjackson 2022-05-12 19:36:42 192.168.247.153
jclark 2022-05-10 10:48:02 192.168.174.117
alevitsk 2022-05-08 12:09:10 192.16874.1390.176
jrafael 2022-05-10 22:40:01 192.168.148.115
yappiah 2022-05-12 10:37:22 192.168.103.10654
daquino 2022-05-08 7:02:35 192.168.168.144
```

Hint 1

Task 6

In this task, you'll build a regular expression pattern that you can use later on to extract IP addresses that are in the form of xxx.xxx.xxx.xxx. In other words, you'll extract all IP addresses that contain four segments of three digits that are separated by periods.

Write a regular expression pattern that will match with these IP addresses and store it in a variable named `pattern`. Use the regular expression symbols `\d` and `\.` in your pattern. Note that the symbol `\d` matches with digits, in other words, any integer between 0 and 9. Be sure to replace the `### YOUR CODE HERE ###` with your own code. Since you'll just build the pattern here, there won't be any output when you run this cell.

In [8]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Assign `pattern` to a regular expression pattern that will match with IP
addresses of the form xxx.xxx.xxx.xxx

pattern = "\d\d\d\.\d\d\d\.\d\d\d\.\d\d\d"
```

Hint 1

Hint 2

Hint 3

Task 7

In this task, you'll use the `re.findall()` function on the regular expression pattern stored in the `pattern` variable and the provided `log_file` to extract the corresponding IP addresses. Afterwards, run the cell and take note of what it outputs. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [12]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Assign `pattern` to a regular expression pattern that will match with IP
addresses of the form xxx.xxx.xxx.xxx

pattern = "\d\d\d\.\d\d\d\.\d\d\d\.\d\d\d"

# Use the `re.findall()` function on `pattern` and `log_file` to extract th
e IP addresses of the form xxx.xxx.xxx.xxx and display the results

print(re.findall(pattern, log_file))
```

```
['192.168.152.148', '192.168.190.178', '192.168.213.128', '192.168.247.15
3', '192.168.174.117', '192.168.148.115', '192.168.103.106', '192.168.168.
144']
```

Hint 1

Hint 2

Question 2

What are some examples of IP addresses that were extracted? What are some examples of IP addresses that were not extracted? Do any that were not extracted seem to be valid IP addresses?

Foram extraídos IPs como 192.168.152.148 e 192.168.190.178, mas IPs como 192.168.22.115 e 192.168.96.200 ficaram de fora; sim, os que ficaram de fora são válidos, mas o padrão atual exige exatamente três dígitos em cada bloco, ignorando segmentos menores.

Task 8

There are some valid IP addresses in the `log_file` that you haven't extracted yet. This is because each segment of digits in a valid IP address can have anywhere between one and three digits.

Adjust the regular expression in the `pattern` to allow for variation in the number of digits in each segment. You can do this by using the `+` symbol after the `\d` symbol. Afterwards, use the updated `pattern` to extract remaining IP addresses. Then, run the cell to analyze the results. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [13]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Update `pattern` to a regular expression pattern that will match with IP
addresses with any variation in the number of digits per segment

pattern = "\d+\.\d+\.\d+\.\d+"

# Use the `re.findall()` function on `pattern` and `log_file` to extract th
e IP addresses of the updated form specifed above and display the results

print(re.findall(pattern, log_file))
```

```
['192.168.152.148', '192.168.22.115', '192.168.190.178', '1923.1689.3.24',
'192.168.213.128', '1924.1680.27.57', '192.168.96.200', '1921.168.1283.7
5', '19245.168.2345.49', '192.168.247.153', '192.168.174.117', '192.16874.
1390.176', '192.168.148.115', '192.168.103.10654', '192.168.168.144']
```

Hint 1

Hint 2

Question 3

What gets extracted here? Do all extracted IP addresses have between one and three digits in every segment?

Todos os endereços que possuem sequências de números separados por pontos foram extraídos, mas nem todos têm entre um e três dígitos por segmento, pois o símbolo + captura qualquer quantidade de números, incluindo IPs inválidos com 4 ou 5 dígitos.

Task 9

Note that all the IP addresses are now extracted but they also include invalid IP addresses with more than three digits per segment.

In this task, you'll update the `pattern` using curly brackets instead of the `+` symbol. In regular expressions, curly brackets can be used to represent an exact number of repetitions between two numbers. For example, `{2,4}` in a regular expression means between 2 and 4 occurrences of something. Applying this to an example, `\w{2,4}` would match with two, three, or four alphanumeric characters. Afterwards, you'll call the `re.findall()` function on the updated `pattern` and the `log_file` and store the output in a variable named `valid_ip_addresses` .

Then, display the contents of `valid_ip_addresses` and run the cell to analyze the results. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [14]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Assign `pattern` to a regular expression that matches with all valid IP a
ddresses and only those

pattern = "\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"

# Use `re.findall()` on `pattern` and `log_file` and assign `valid_ip_addre
sses` to the output

valid_ip_addresses = re.findall(pattern, log_file)

# Display the contents of `valid_ip_addresses`

print(valid_ip_addresses)
```

```
['192.168.152.148', '192.168.22.115', '192.168.190.178', '192.168.213.12
8', '192.168.96.200', '192.168.247.153', '192.168.174.117', '192.168.148.1
15', '192.168.103.106', '192.168.168.144']
```

Hint 1

Hint 2

Question 4

What do you notice about the extracted IP addresses here compared to those extracted in the previous two tasks?

Diferente das tarefas anteriores, agora apenas IPs válidos foram extraídos, pois o uso de {1,3} limitou cada segmento a no máximo três dígitos, descartando automaticamente os endereços malformados que apareceram na Tarefa 8.

Task 10

Now, all of the valid IP addresses have been extracted. The next step is to identify flagged IP addresses.

You're given a list of IP addresses that have been previously flagged for unusual activity, stored in a variable named `flagged_addresses`. When these addresses are encountered, they should be investigated further. This list is just for educational purposes and contains examples of private IP addresses that are found only within internal networks.

Display this list and examine what it contains by running the cell. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [15]:
```
# Assign `flagged_addresses` to a list of IP addresses that have been previ
ously flagged for unusual activity

flagged_addresses = ["192.168.190.178", "192.168.96.200", "192.168.174.11
7", "192.168.168.144"]

# Display the contents of `flagged_addresses`

print(flagged_addresses)
```

```
['192.168.190.178', '192.168.96.200', '192.168.174.117', '192.168.168.14
4']
```

Hint 1

Task 11

Finally, you will write an iterative statement that loops through the `valid_ip_addresses` list and checks if each IP address is flagged. In the following code, the `address` will be the loop variable. Also, include a conditional that checks if the `address` belongs to the `flagged_addresses` list. If so, it should display `"The IP address _____ has been flagged for further analysis."` If not, it should display `"The IP address _____ does not require further analysis."` Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [17]:
```
# Assign `log_file` to a string containing username, date, login time, and
IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09
6:46:40 192.168.22.115 \nsmartell 2022-05-09 19:30:32 192.168.190.178 \naru
tley 2022-05-12 17:00:59 1923.1689.3.24 \nrjensen 2022-05-11 0:59:26 192.16
8.213.128 \naestrada 2022-05-09 19:28:12 1924.1680.27.57 \nasundara 2022-05
-11 18:38:07 192.168.96.200 \ndkot 2022-05-12 10:52:00 1921.168.1283.75 \na
bernard 2022-05-12 23:38:46 19245.168.2345.49 \ncjackson 2022-05-12 19:36:4
2 192.168.247.153 \njclark 2022-05-10 10:48:02 192.168.174.117 \nalevitsk 2
022-05-08 12:09:10 192.16874.1390.176 \njrafael 2022-05-10 22:40:01 192.16
8.148.115 \nyappiah 2022-05-12 10:37:22 192.168.103.10654 \ndaquino 2022-05
-08 7:02:35 192.168.168.144"

# Assign `pattern` to a regular expression that matches with all valid IP a
ddresses and only those

pattern = "\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"

# Use `re.findall()` on `pattern` and `log_file` and assign `valid_ip_addre
sses` to the output

valid_ip_addresses = re.findall(pattern, log_file)

# Assign `flagged_addresses` to a list of IP addresses that have been previ
ously flagged for unusual activity

flagged_addresses = ["192.168.190.178", "192.168.96.200", "192.168.174.11
7", "192.168.168.144"]

# Iterative statement begins here
# Loop through `valid_ip_addresses` with `address` as the loop variable

for address in valid_ip_addresses:

    # Conditional begins here
    # If `address` belongs to `flagged_addresses`, display "The IP address
_____ has been flagged for further analysis."

    if address in flagged_addresses:
        print("The IP address", address, "has been flagged for further anal
ysis.")

    # Otherwise, display "The IP address _____ does not require further an
alysis."

    else:
        print("has been flagged for further analysis")
```

```
has been flagged for further analysis
has been flagged for further analysis
The IP address 192.168.190.178 has been flagged for further analysis.
has been flagged for further analysis
The IP address 192.168.96.200 has been flagged for further analysis.
has been flagged for further analysis
The IP address 192.168.174.117 has been flagged for further analysis.
has been flagged for further analysis
has been flagged for further analysis
The IP address 192.168.168.144 has been flagged for further analysis.
```

Hint 1

Hint 2

Hint 3

Conclusion

What are your key takeaways from this lab?

Este laboratório demonstrou para mim como expressões regulares automatizam a análise de logs, permitindo filtrar IDs de dispositivos específicos e extrair endereços IP com precisão através do uso de metacaracteres como \w, \d, e quantificadores como + e {n,m} para validar formatos de dados.